

21. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

/* -----
MODULE 1: Input Handling
----- */

int readInput(char *input, int size) {
    printf("shell> ");
    fflush(stdout);

    if (fgets(input, size, stdin) == NULL) {
        return 0;
    }

    input[strcspn(input, "\n")] = '\0';
    return 1;
}

/* -----
MODULE 2: Command Parsing
----- */

int parseCommand(char *input, char *args[]) {
    int count = 0;
    char *token = strtok(input, " ");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count] = token;
        count++;
        token = strtok(NULL, " ");
    }
    args[count] = NULL;
    return count;
}

/* -----
MODULE 3: Interface Validation
----- */

int validateCommand(char *args[], int count) {
    if (count == 0) {
```

```
/* -----
MODULE 3: Interface Validation
----- */

int validateCommand(char *args[], int count) {
    if (count == 0) {
        printf("Error: Empty command.\n");
        return 0;
    }

    if (args[0] == NULL) {
        printf("Error: Invalid command interface.\n");
        return 0;
    }

    return 1;
}

/* -----
MODULE 4: Command Execution
----- */

int executeCommand(char *args[]) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 0;
    }

    /* Child process */
    if (pid == 0) {
        execvp(args[0], args);

        /*
         * If execvp returns,
         * execution failed.
         */
        perror("Command execution failed");
        exit(1);
    }

    /* Parent process */
    int status;
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid failed");
        return 0;
    }

    if (WIFEXITED(status)) {
```

```

/* Parent process */
int status;
if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid failed");
    return 0;
}

if (WIFEXITED(status)) {
    int exitStatus = WEXITSTATUS(status);
    if (exitStatus == 0) {
        return 1;
    }

    printf("Command exited with status %d\n",
        exitStatus);
    return 0;
}

return 0;
}

/* -----
MODULE 5: Command Processing
----- */

void processCommand(char *input) {
    char *args[MAX_ARGS];

    /* Parse input.
    */
    int count = parseCommand(input, args);

    /* Validate interface between parser
    * and executor.
    */
    if (!validateCommand(args, count)) {
        return;
    }

    /* Handle exit command.
    */
    if (strcmp(args[0], "exit") == 0) {
        printf("Exiting shell...\n");
        exit(0);
    }

    /* Execute command.
    */
}

```

```

/* Execute command.
*/
if (executeCommand(args)) {
    printf("Command executed successfully.\n");
} else {
    printf("Command execution failed.\n");
}
}

/* -----
MAIN MODULE
----- */

int main() {
    char input[MAX_INPUT];

    printf("Modular Shell Integration System\n");
    printf("-----\n");

    printf("Type commands such as:\n");
    printf("  echo Hello\n");
    printf("  pwd\n");
    printf("  ls\n");
    printf("  exit\n\n");

    while (1) {
        /* Module 1:
        * Read user input.
        */
        if (!readInput(input, MAX_INPUT)) {
            printf("\nInput error or EOF.\n");
            break;
        }

        /* Ignore empty input.
        */
        if (strlen(input) == 0) {
            continue;
        }

        /* Modules 2-5:
        * Parse → Validate → Execute
        */
        processCommand(input);
    }

    return 0;
}

```

Result

```

ketana_sindhu@KetaZone:~$ nano Skill21a.c
ketana_sindhu@KetaZone:~$ gcc Skill21a.c -o s21a
ketana_sindhu@KetaZone:~$ ./s21a
Modular Shell Integration System
-----
Type commands such as:
  echo Hello
  pwd
  ls
  exit

shell> echo Hello
Hello
Command executed successfully.
shell> pwd
/home/ketana_sindhu
Command executed successfully.
shell> ls
AddressTranslation.c  FirstFile.c      SC              Skill1.c         WeekTwoONE.c    fork.i          p              s20b
DemandPage.c         Fork.c           SG              Skill18b.c       a.out           fork.o          p3             s21a
DisplayProcessID.c   HELLO.c         SKILL           Skill19a.c       ad              fork.s          p4             s9a
FILE.C               KL.c            SP              Skill19b.c       address         fork_demo.c     parent         seg
FR                   MemoryAllocate.c SS              Skill20a.c       cow             hello           pid            shortcut
FW                   PG              ST              Skill20a.c.save  cow.c           ketana         pipe.c          signals.c
Fifo                 PID.c           Segmentation.c  Skill20b.c       demand         ketana_sindhu  pipe.c.save    sindhu
Fifo.c               ParentAndChild.c SigChild.c       Skill21a.c       example.txt     malloc          priority.c     sjfs
FifoRead.c           PipeGrip.c       SigPipe.c        Variable.c       fcfs.c          memory          rr             sjfs.c
FifoWrite.c          Practical3.c      SigTerm.c        Virtual.c        fcfs.c          memory.c        s              variable
File                  Practical4.c      Signals.c         Virtual.c.save   file            memory_alloc    s18b           virtual
File.c               ReadUser.c       Sigsegu.c        WAIT             file.c          memory_alloc.c  s19a           wait
FirstFile            RoundRobin.c     Sindhu           Wait.c           fork            myfifo          s19b           wait.c
FirstFile            S2Commands       Sindhu.c         WeekTwo.c        fork.c          name            s20a
Command executed successfully.
shell> exit
Exiting shell...
ketana_sindhu@KetaZone:~$

```

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

/* -----
Log error messages
----- */
void logError(const char *message) {
    FILE *logFile = fopen("shell_errors.log", "a");
    if (logFile == NULL) {
        perror("Unable to open error log");
        return;
    }
    fprintf(logFile, "ERROR: %s\n", message);
    fclose(logFile);
}

/* -----
Display and log error
----- */
void reportError(const char *message) {
    fprintf(stderr, "Error: %s\n", message);
    logError(message);
}

/* -----
Parse command
----- */
int parseCommand(char *input, char *args[]) {
    int count = 0;
    char *token = strtok(input, " \t");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count] = token;
        count++;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;
    return count;
}

```

```

/* -----
Validate syntax
----- */
int validateSyntax(char *input) {
    if (input == NULL || strlen(input) == 0) {
        reportError("Empty command");
        return 0;
    }
    /* Check for invalid pipe at beginning */
    if (input[0] == '|') {
        reportError("Invalid syntax: command cannot start with |");
        return 0;
    }
    /* Check for invalid pipe at end */
    int length = strlen(input);
    if (input[length - 1] == '|') {
        reportError("Invalid syntax: command cannot end with |");
        return 0;
    }
    /* Check consecutive pipes */
    if (strstr(input, "||") != NULL) {
        reportError("Invalid syntax: consecutive pipes");
        return 0;
    }
    return 1;
}

/* -----
Execute command
----- */
int executeCommand(char *args[]) {
    pid_t pid = fork();
    if (pid < 0) {
        reportError("Runtime error: fork failed");
        return 0;
    }
    if (pid == 0) {

```

```

        /* Execute command.
        */
        execvp(args[0], args);
        /* If execvp returns, command failed.
        */
        fprintf(stderr,
            "Runtime Error: command '%s' not found\n",
            args[0]);
        exit(127);
    }
    /* Parent waits for child */
    int status;
    if (waitpid(pid, &status, 0) == -1) {
        reportError("Runtime error: waitpid failed");
        return 0;
    }
    /* Check exit status */
    if (WIFEXITED(status)) {
        int exitStatus = WEXITSTATUS(status);
        if (exitStatus == 0) {
            return 1;
        }
        char message[100];
        snprintf(message,
            sizeof(message),
            "Command exited with status %d",
            exitStatus);
        logError(message);
        return 0;
    }
    if (WIFSIGNALED(status)) {
        char message[100];
        snprintf(message,
            sizeof(message),
            "Command terminated by signal %d",
            WTERMSIG(status));
        logError(message);
        return 0;
    }

```

```

    }
    return 0;
}

/* -----
Process command
----- */
int processCommand(char *input) {
    char *args[MAX_ARGS];
    /* Validate syntax */
    if (!validateSyntax(input)) {
        return 0;
    }
    /* Parse command */
    int count = parseCommand(input, args);
    if (count == 0) {
        reportError("No command entered");
        return 0;
    }
    /* Exit command */
    if (strcmp(args[0], "exit") == 0) {
        printf("Exiting shell...\n");
        return 2;
    }
    /* Execute command */
    if (executeCommand(args)) {
        printf("Command executed successfully.\n");
    } else {
        printf("Command failed, but shell recovered.\n");
    }
    return 1;
}

/* -----
Main
----- */
int main() {
    char input[MAX_INPUT];
    printf("Runtime Error Handling Shell\n");

```

```

    /* Execute command */
    if (executeCommand(args)) {
        printf("Command executed successfully.\n");
    } else {
        printf("Command failed, but shell recovered.\n");
    }
    return 1;
}

/* -----
Main
----- */
int main() {
    char input[MAX_INPUT];

    printf("Runtime Error Handling Shell\n");
    printf("-----\n");

    printf("Try commands such as:\n");
    printf("  echo Hello\n");
    printf("  pwd\n");
    printf("  invalidcommand\n");
    printf("  | invalid\n");
    printf("  invalid |\n");
    printf("  exit\n");

    while (1) {
        printf("shell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL) {
            reportError("Failed to read input");
            break;
        }

        input[strcspn(input, "\n")] = '\0';
        int result = processCommand(input);
        if (result == 2) {
            break;
        }
    }

    printf("\nError handling test completed.\n");
    return 0;
}

```

Result:

```
saisharan134@APL-5CD1366K5L:~/SKILL$
saisharan134@APL-5CD1366K5L:~/SKILL$
saisharan134@APL-5CD1366K5L:~/SKILL$
Runtime Error Handling Shell
-----
Try commands such as:
  echo Hello
  pwd
  invalidcommand
  | invalid
  invalid |
  exit

shell> echo Hello
Hello
Command executed successfully.
shell> pwd
saisharan134@APL-5CD1366K5L:~/SKILL$
Command executed successfully.
shell> | invalid
Error: Invalid syntax: command cannot start with |
shell> invalidcommand
Runtime Error: command 'invalidcommand' not found
Command failed, but shell recovered.
shell> exit
Exiting shell...

Error handling test completed.
saisharan134@APL-5CD1366K5L:~/SKILL$~$ |
```